Deep State Space Models: From Convolutions to S4, S5, and Mamba

Ethan Read

May 12, 2026

Contents

Preface

These notes are written for a reader who already understands ordinary RNNs, transformers, backpropagation, regularization, and standard deep-learning training practice. The goal is to rebuild the missing bridge from convolutional sequence models to modern deep state space models: S4 (Gu et al., 2022), S5 (Smith et al., 2023), and Mamba (Gu and Dao, 2023).

The intended level is graduate-style: enough linear algebra, signal-processing language, and implementation detail to read the original papers directly, while keeping derivations in service of model understanding. The final chapter connects the architecture ideas back to self-supervised Utah array sequence modeling.

Chapter 1

Convolutions For Sequences

Purpose

This chapter rebuilds convolution from the sequence-modeling point of view. The target is not image CNN fluency, but the set of convolution ideas needed before state space models: causal filters, receptive fields, dilation, channel mixing, and the fact that linear time-invariant recurrences can be evaluated as convolutions.

Assumed Background

The reader already understands feed-forward neural networks, backpropagation, tensor shapes, minibatching, and why transformers parallelize over positions while vanilla RNNs do not.

Key Concepts To Define

- Discrete convolution and cross-correlation, with consistent indexing conventions.
- Kernels, channels, padding, stride, receptive field, and boundary effects.
- Causal convolution as the sequence-modeling version of "no future leakage."
- Dilated convolution and exponential receptive-field growth.
- Depthwise, grouped, and pointwise convolutions as different forms of channel mixing.
- FFT convolution and why long convolution kernels can be applied in parallel.
- CNN sequence blocks as finite-memory filters, contrasting them with RNN hidden states.

Core Equations To Develop

For an input sequence $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_L$ and a causal kernel $\boldsymbol{K}_0, \ldots, \boldsymbol{K}_{K-1}$, the basic sequence convolution is

$$\boldsymbol{y}_t = \sum_{i=0}^{K-1} \boldsymbol{K}_i \boldsymbol{u}_{t-i},$$

where out-of-range inputs are treated according to the padding convention. This equation will become the bridge to SSMs, where the kernel is not a short learned CNN filter but an implicit long filter generated from state-space parameters.

Primary Sources

Use the convolution chapters of Goodfellow et al. (2016) and the CS231n convolutional network notes (Li et al., 2024) for basic CNN vocabulary. The chapter should reinterpret those ideas for one-dimensional temporal data rather than image grids.

Prepares You For

The next chapter treats convolutions as filters generated by linear dynamical systems. The important mental shift is that a convolution kernel can be either directly learned, as in a CNN, or indirectly generated, as in S4 and S5.

Chapter 2

Linear Systems And Filters

Purpose

This chapter refreshes the linear algebra and signal-processing language behind state space models. It should make precise why a linear recurrence has an impulse response, why that impulse response defines a convolution, and why stability depends on eigenvalues.

Assumed Background

The reader remembers matrix multiplication, eigenvalues at a high level, and ordinary RNN recurrences, but may need a careful restart on diagonalization, complex eigenvalues, and stability.

Key Concepts To Define

- Linear recurrence, input, state, output, and hidden dimension.
- Impulse response and finite versus infinite impulse-response filters.
- Stability in discrete time: why eigenvalues inside the unit circle matter.
- Diagonalization and modal decomposition.
- Complex conjugate eigenpairs and oscillatory modes.
- Transfer functions as a compact description of filter behavior.
- Vanishing, exploding, and slowly decaying memory in linear recurrences.

Core Equations To Develop

Start from the linear recurrence

$$\boldsymbol{x}_t = \boldsymbol{A}\boldsymbol{x}_{t-1} + \boldsymbol{B}\boldsymbol{u}_t, \qquad \boldsymbol{y}_t = \boldsymbol{C}\boldsymbol{x}_t + \boldsymbol{D}\boldsymbol{u}_t.$$

Unrolling gives terms involving $\boldsymbol{C}\boldsymbol{A}^k\boldsymbol{B}$, which form an impulse-response kernel. This is the simplest version of the convolutional view used by deep SSM layers.

Prepares You For

The next chapter formalizes state space models and explains how continuous-time dynamics are discretized for neural network sequence layers.

Chapter 3

State Space Models

Purpose

This chapter introduces state space models as a general architecture template rather than only as classical control models. It should connect continuous dynamics, discrete sequence layers, recurrent evaluation, and convolutional training.

Assumed Background

The reader knows RNNs and transformers, but may not know control-theory notation or the continuous-time origin of SSM layers.

Key Concepts To Define

- Continuous-time SSMs with state $\boldsymbol{x}(t)$, input $\boldsymbol{u}(t)$, and output $\boldsymbol{y}(t)$.
- Discrete-time SSMs with matrices $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}, \boldsymbol{D}$.
- State size versus model width versus input and output channel count.
- Discretization, step size Δ, and zero-order hold.
- Recurrent mode for generation or online inference.
- Convolution mode for parallel training.
- Single-input single-output versus multi-input multi-output SSMs.

Core Equations To Develop

The continuous model is

$$\frac{d}{dt}\boldsymbol{x}(t) = \boldsymbol{A}\boldsymbol{x}(t) + \boldsymbol{B}\boldsymbol{u}(t), \qquad \boldsymbol{y}(t) = \boldsymbol{C}\boldsymbol{x}(t) + \boldsymbol{D}\boldsymbol{u}(t).$$

After discretization, the layer can be run recurrently or converted into a convolution kernel

$$\boldsymbol{K}_k = \boldsymbol{C}\bar{\boldsymbol{A}}^k\bar{\boldsymbol{B}},$$

where bars denote discretized parameters.

Prepares You For

The next chapter explains why modern SSMs needed special initializations and structured state matrices to store long histories without becoming unstable or computationally expensive.

Chapter 4

HiPPO And Memory

Purpose

This chapter explains HiPPO as the memory mechanism that made long-range SSM initialization plausible. The goal is not to reproduce every derivation, but to understand what problem HiPPO solves and why it matters for S4 and S5.

Assumed Background

The reader has seen polynomial bases before but may need reminders about projection, orthogonality, and approximation of functions by basis coefficients.

Key Concepts To Define

- Online memory as compression of a growing input history.
- Function approximation by basis coefficients.
- Orthogonal polynomials and Legendre bases at a conceptual level.
- Projection of the recent past into a finite-dimensional state.
- Why naive RNN memory is hard to initialize for long contexts.
- How HiPPO-derived matrices become SSM initializations.

Primary Source

The main source is Gu et al. (2020). The chapter should separate the high-level memory idea from the more technical measure-specific formulas.

Prepares You For

S4 uses structured versions of HiPPO-inspired state matrices to get both long memory and efficient computation.

Chapter 5

S4

Purpose

This chapter explains S4 as the first major modern deep SSM layer that made long-sequence modeling competitive. It should answer two questions: what does S4 compute, and what structure makes it efficient?

Assumed Background

The reader has completed the prior chapters on convolutions, linear systems, SSMs, and HiPPO. RNNs and transformers are already familiar reference points.

Key Concepts To Define

- The long-range arena motivation and why ordinary sequence models struggled.
- Structured state matrices and normal plus low-rank structure.
- SISO SSM copies and channel mixing around the SSM layer.
- Cauchy kernel computation at a conceptual level.
- Convolutional training versus recurrent inference.
- Stability, initialization, and parameterization of complex-valued dynamics.
- The S4 block as used inside a deep network.

Primary Source

The main source is Gu et al. (2022). This chapter should prioritize the architectural logic and computation path over implementation-specific kernel tricks.

Prepares You For

S5 keeps the state-space viewpoint but simplifies the parameterization and changes the multi-channel treatment.

Chapter 6

S5

Purpose

This chapter explains S5 as a simpler, practical SSM layer that is especially relevant to the Utah SSL project. It should clarify what S5 inherits from S4 and what it changes.

Assumed Background

The reader understands the SSM convolution kernel, diagonalization, and why S4 used structured state matrices.

Key Concepts To Define

- Diagonal state matrix parameterization.
- Multi-input multi-output SSMs instead of many independent SISO SSMs.
- Parallel scan as an alternative to explicit convolution kernels.
- Initialization from S4/HiPPO-style dynamics.
- State size, model width, and channel mixing in S5 blocks.
- Why S5 can be attractive for neural time series and causal decoding.

Primary Source

The main source is Smith et al. (2023). Project-specific interpretation can draw on the local S5 notes in `docs/notes`, but this learning chapter should remain broadly educational.

Prepares You For

Mamba modifies the SSM idea by making the dynamics selective, meaning some SSM parameters depend on the current input.

Chapter 7

Mamba

Purpose

This chapter explains Mamba as a selective state space model: still linear-time and recurrent in spirit, but no longer a fixed linear time-invariant convolution.

Assumed Background

The reader understands SSMs, convolutional training for time-invariant systems, and the transformer idea of content-dependent routing through attention.

Key Concepts To Define

- Selection as input-dependent SSM parameters.
- Why fixed LTI convolution cannot implement content-based routing.
- Input-dependent $\boldsymbol{B}$, $\boldsymbol{C}$, and step size Δ.
- Why selectivity breaks ordinary convolution mode.
- Hardware-aware parallel scan.
- The Mamba block: projection, local convolution, selective scan, gating, and output projection.
- Differences between attention, S4/S5, and Mamba for long sequences.

Primary Sources

The main source is Gu and Dao (2023). The optional follow-up is Mamba-2 and structured state space duality (Dao and Gu, 2024), which can be introduced after the original Mamba chapter is solid.

Prepares You For

The final comparison chapter places Mamba alongside CNNs, RNNs, transformers, S4, and S5, with special attention to Utah array self-supervised learning.

Chapter 8

Comparison And Project Relevance

Purpose

This chapter turns the technical material into modeling judgment. The goal is to compare architecture families and make clear why S5 and Mamba are plausible backbones for self-supervised neural decoding.

Architectures To Compare

- CNNs: fixed finite filters with strong parallelism.
- RNNs: recurrent hidden states with simple online inference but limited parallelism.
- Transformers: content-based global mixing with high memory cost at long context.
- S4: structured long convolution kernels generated by SSM parameters.
- S5: simplified diagonal MIMO SSMs with efficient parallel scan.
- Mamba: selective SSMs with input-dependent dynamics and linear-time scaling.

Comparison Axes

- Parallel training and online inference.
- Long-range memory and stability.
- Content-dependent routing versus fixed dynamics.
- Parameter efficiency and state size.
- Causal operation and future leakage.
- Suitability for Utah array SSL objectives such as future prediction and masked reconstruction.

Project Connection

The local project has already identified S5 and Mamba as the main SSM-family candidates for causal self-supervised Utah array modeling. This chapter should connect the theory to practical choices: raw-bin versus causal-patch inputs, subject/session adaptation, frozen phoneme probes, and whether SSL metrics reflect speech-relevant structure rather than recording-context shortcuts.

Exit Criteria For The Notes

After finishing the full document, the reader should be able to read the S4, S5, and Mamba papers without getting stuck on core notation, understand the computational tradeoffs, and reason about which model family is appropriate for a causal neural decoder.

Bibliography

Dao, T. and Gu, A. (2024). Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

Gu, A. and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.

Gu, A., Dao, T., Ermon, S., Rudra, A., and Ré, C. (2020). HiPPO: Recurrent memory with optimal polynomial projections. In *Advances in Neural Information Processing Systems*.

Gu, A., Goel, K., and Ré, C. (2022). Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*.

Li, F.-F., Johnson, J., and Yeung, S. (2024). CS231n: Convolutional neural networks for visual recognition. Stanford University course notes.

Smith, J. T. H., Warrington, A., and Linderman, S. W. (2023). Simplified state space layers for sequence modeling. In *International Conference on Learning Representations*.